Moving from Quality Assurance to Quality Assistance

Óscar Barrios
Test Automation Specialist @SUSE

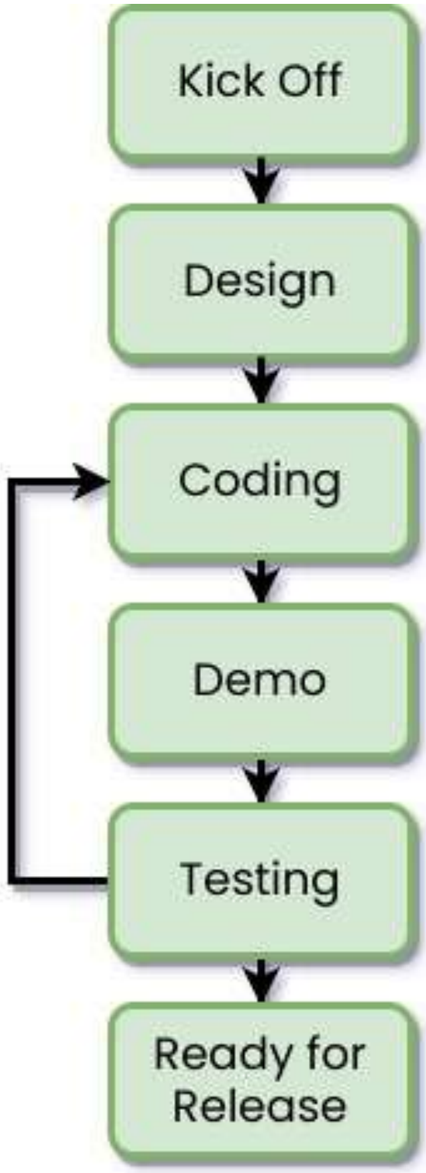
Quality Assurance

It’s the process of verifying whether a product meets the required specifications

Quality Assurance model

Testers are mostly involved in the last stages of the process.

Their main goal is to find bugs and tell the project manager if there is no critical issue before releasing.



Kick off

This is a short catchup between the **Product Owner and the development team**, to kick off the implementation of the story. The aim of this is to ensure everyone involved is on the same page regarding the expectations from this story.

The PO explains the story from user's point of view. In some cases, the PO might only know the need, so the team must find the right way to satisfy this need.

During this stage, it's expected to clarify the main scenarios, so we can end with an **agreement on acceptance criteria**, clarifying the expected outcome.

Design

On that phase, the developers discusss how must be designed the new feature. Taking into consideration the **technology to use**, the **integration with other features** in the product, **usability**, performance, etc.

The outcome from this phase will be written, for instance in form of a RFC or inside the main card identifying the feature in your board.

Coding

As the name suggests, this is where the developers code the implementation of the story.

Along with the implementation, unit tests are added too.

Demo

Once the developer is confident with the implementation, we will organize a session to **demonstrate the story in action**.

The team can then ask questions about the scenarios, trying any corner case they have in mind. This step often **helps find low hanging fruit**, and the developers are able to make quick changes to fix these, as the code is still fresh in their minds.

This is often the first stage where a **Quality Assurance Engineer is involved**.

Testing

Following a successful demo, the software is **merged into the master branch and deployed into a Test environment**.

Here, the Tester tests the story, mostly using exploratory testing techniques. If any important **bugs are found**, the story gets **sent back to the Developer** for fixing. Once the story is tested successfully, the story is now ready to release.

Problems

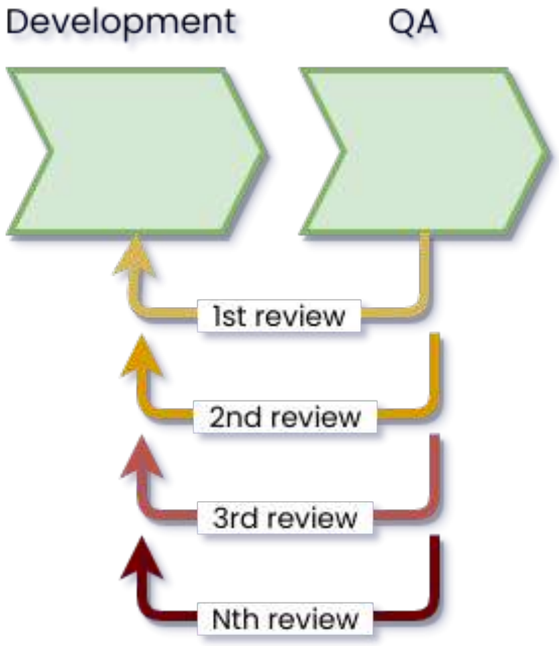
This process can work very well for small or new projects. But it has difficulties to scale.

Many stories, while in the Testing stage, go through a cycle of bug fixes and retest. Whilst it's great that bugs get caught in testing phase rathen than going to production, to **find them earlier will prevent some inefficiency** in the development process:

- Implies an extra of time, **fixing** the software and **retesting** it.
- Often, the **tester is busy with other stories**, and the **developer has to move on to another story**, and they will both need to context switch to this bug. This is disruptive to a continuous flow of work.

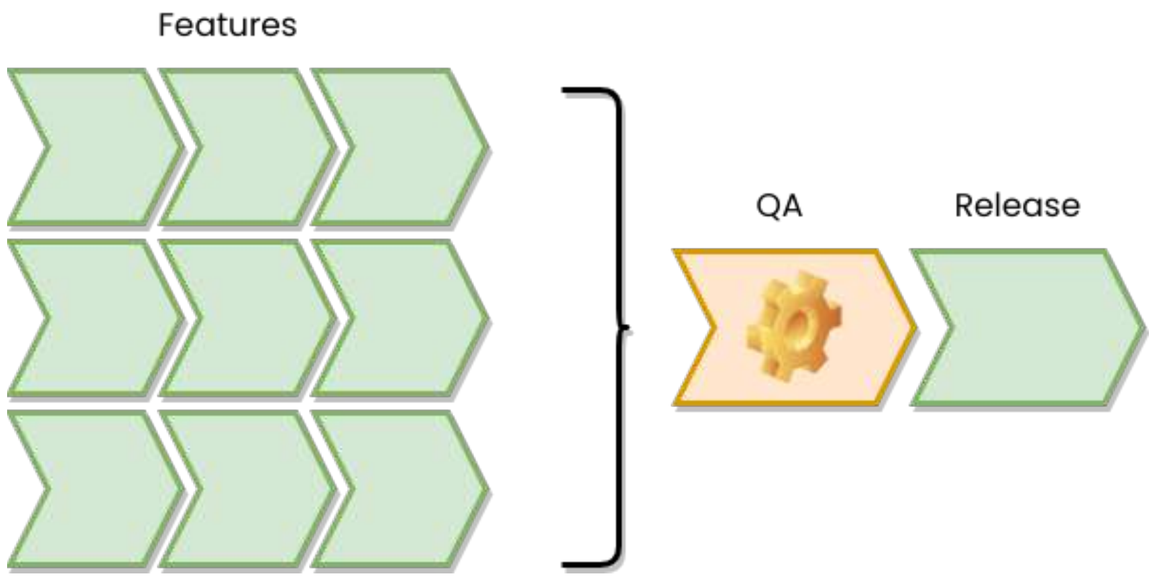
Ping-pong problem

When the QA team tests a new feature, some bugs may arise. User stories go back to the developers who have to fix them. After, the user stories return to the QA team, and some new bugs may arise again. It can be an endless loop.



Bottleneck problem

QA team is a bottleneck when they are at the end of the process. If the user stories are sent to test altogether, or the QA team is not well balanced; we need to wait for the “ok” of the QA team. The feature can take a bit of time to reach production.



Quality Assistance

It helps developers build quality into their features from the design to the release

Principles

“The team should focus
on **prevention** rather than detection.”

“We **involve the QA team** in earlier
phases of the product process.”

“If team members believe that QA sho
uld be catching the bugs, the model wi
ll break down.”

Benefits of the Quality Assistance model

As the team moves further towards this model, you should aim to see improvements in these three areas:

- Quality
- Efficiency
- Empowerment

Quality

Involving a QA professional in an early stage of the product, enforce that **quality is built in at every stage of the software delivery life cycle**.

Furthermore, due to collaboration and sharing of insights, a developer's knowledge and **understanding of quality aspects will increase**, which will result in them writing better quality software.

Efficiency

Preventing bugs before they are written, catching them before the developer change of context and having **faster feedback**. Therefore, we can **ship faster** and have **confidence** that the quality remains high.

The main driver of efficiency in the Quality Assistance approach lies in **automation**, not just test automation, but embracing DevOps culture in all the stages.

Furthermore, the **time savings mean that QA can carry out more complex testing** within their timeframe, which results in increased software quality.

Empowerment

As development and Quality Assistance efforts work in synergy, **developers can feel confident** in the quality of software they're producing.

QA professionals will also have **more scope to focus on the root causes of quality issues and their symptoms**, as well as outline strategies to overcome them going forward.

Interim techniques

Changing an existing team's quality process will require time and effort, you're requiring the whole team to be responsible for the quality of the team's output instead of relying on QA to be gatekeepers

QA in Kickoffs

Pairing with developers **encourages them to think of all the risks and edge cases upfront**, before they start implementing the story.

You can brainstorm together to come up with acceptance tests and sort out any ambiguities from the start.

Remark:

Reliance on QA in Kickoffs means that the developers require a QA engineer to come up with potential problems before coding, rather than being able to come up with them by themselves.

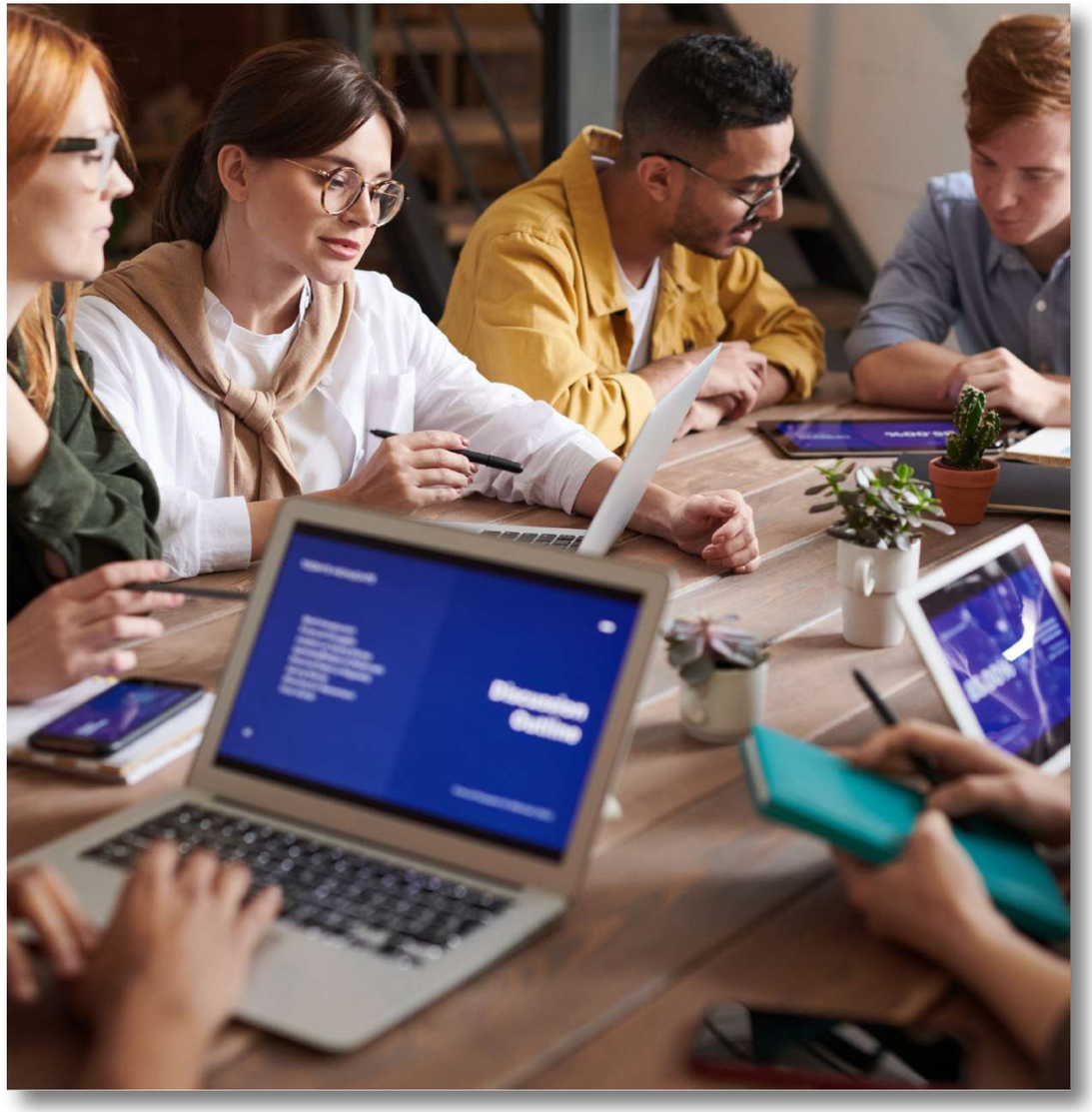
Blitz Testing

Asking the **whole team to join in a time-boxed testing session** is a fun way for teams to check that they are confident with the features they are shipping.

We can perform these sessions, as an interim tecnique, so **the whole team gains test expertise thinking possible corner cases before even design the feature**, it could also help to understand the whole product.

Remark:

A mature development team, should be confindent on what they deliver, so we should not rely on this sessions in long-term.



Dogfooding

Running your unreleased features on internal instances is a **good way to validate that they are useful and easy to use**.

We must **treat them as production** instances, and see shipping a bug as a failure of the testing process.

Remark:

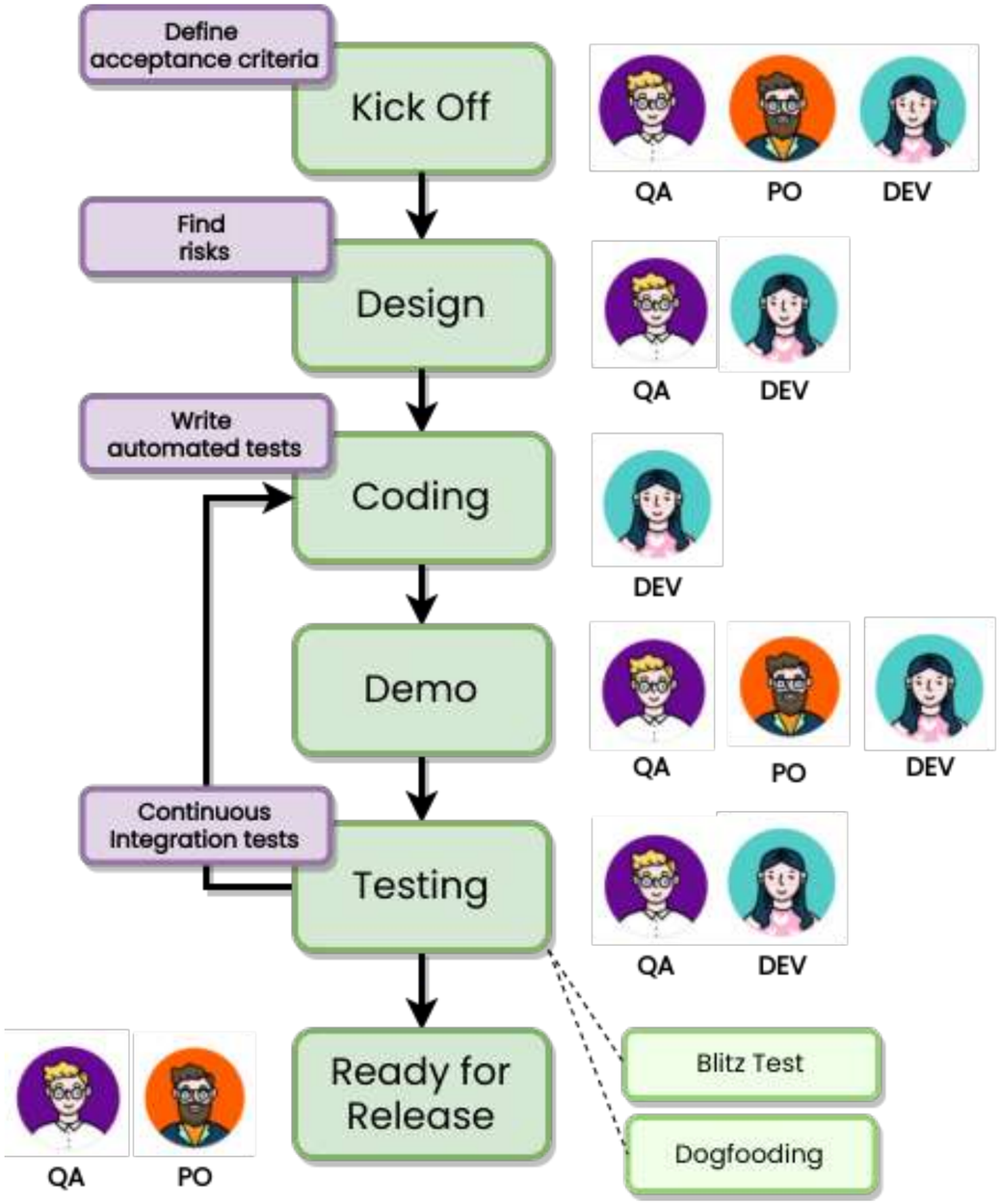
Treating dogfooding as a necessary step to find all the bugs is a sign that the team is not confident in their own implementation, think it twice, in that case, find how to improve previous development phases.

Quality Assistance model

We involve the QA team in early phases of the product process. They participate in the design phase to make sure we have covered all flows and scenarios. This initial involvement helps the team to have better-defined features.

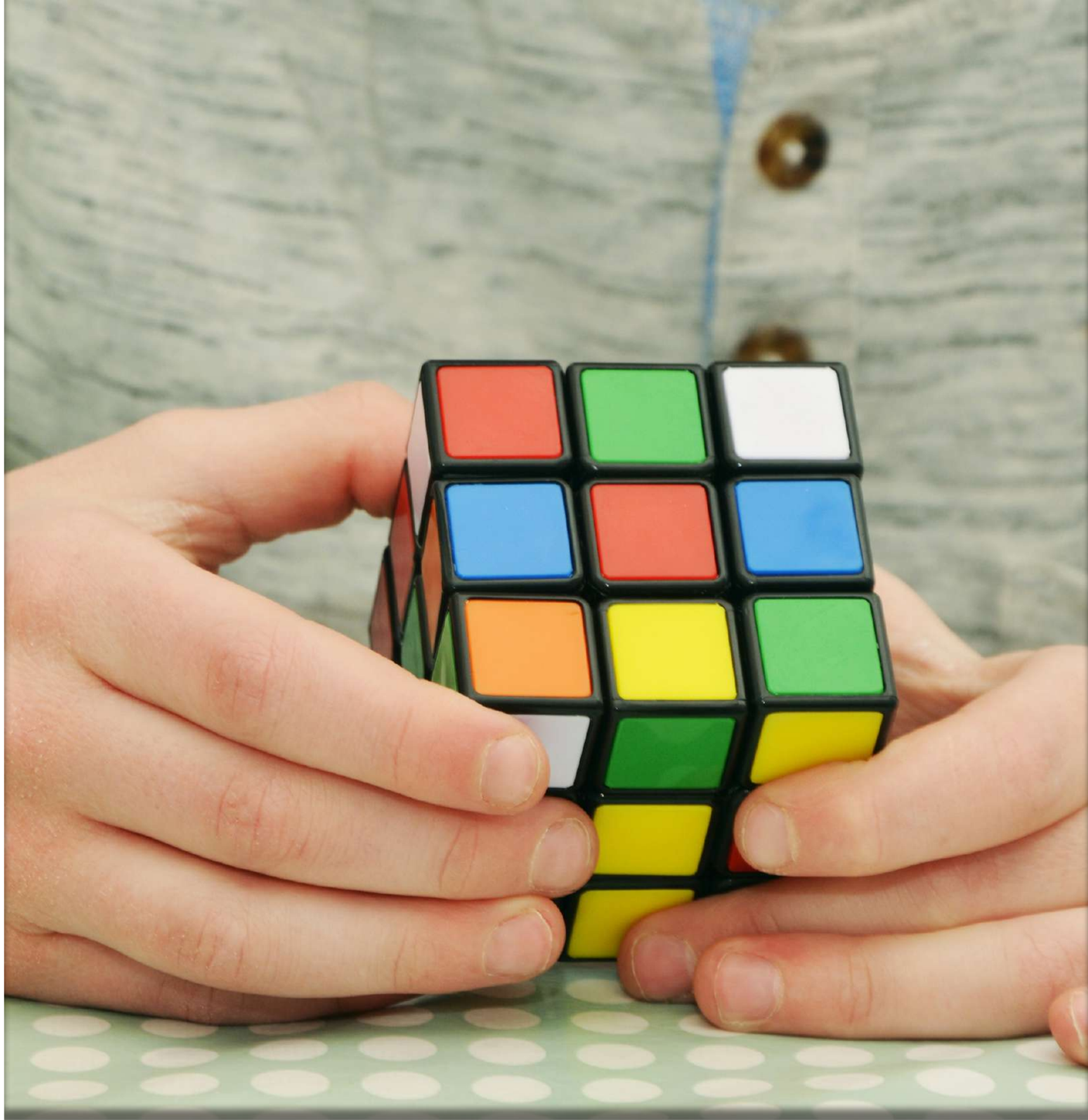
Experimentation is the keyword.

Be patient, iterate,
and find the workflow that fits in your team.



Quality Assistance skills

- Influencing
- Providing testing expertise
- Educating
- Inspiring
- Identifying problems
- Facilitating



Influencing

You will need to **convince everybody** on the team **to embrace whole-team responsibility for quality**. Testing must become part of the definition of done of a story, but this is not just about testing, this is about quality, you can improve the quality by influencing on design, sharing architecture best practices, development methodologies like ATDD or BDD, and many other areas.

Tip:

You should make sure that the input you are giving is valuable. **Don't give your opinion**: it's no more valuable than anyone else's just because you come from QA. Instead, **provide** the team with **facts**.

Providing testing expertise

Developers are experts at coding, but may not have much experience when it comes to testing. You must be able to give accurate and valuable input for high-quality development of individual features and stories. Developers will tests their features, but you must facilitate tools for that purpose.

How to practice

A skilled QA Engineer is able to **quickly identify the risks for a particular story**. This comes from an understanding of the implementation of a feature, and often the developer is the best source of that information. You can become more effective by **asking the developer to explain the implementation details**, and by asking questions about **how it would handle concrete scenarios**. Exploratory testing is a valuable technique, which takes practice.

Educating

Think back to how you learned to test, and pass that experience on. Design workshops and practice sessions for your developers. Be willing to give advice, pair with them, and answer questions when they have doubts.

It's all about finding defects in the implementation as soon as possible, this is not about automate a test following the expected steps from an user, those of course we expecte to pass! So, educate the team to embrace this mindset, open your mind, find all possible scenarios that a user could go throught.

Inspiring

Share your motivations with the team.

Make sure they don't view testing as mindlessly clicking around a UI, hoping to stumble across bugs.

If developers fall into that trap they'll resent the activity as a waste of time, and perform it poorly. So, challange them with some automation techniques, show them the value of having fast feedback, inspire them to think in corner cases, acting like an user, thinking on scalability on security, there is a huge world behind quality!

Identifying problems

While the developers perform the actual testing, you will be looking out for problems and creating opportunities to improve the team's quality, efficiency, and empowerment.

Become a champion of the changes you think are most important for your team and lead their implementation.

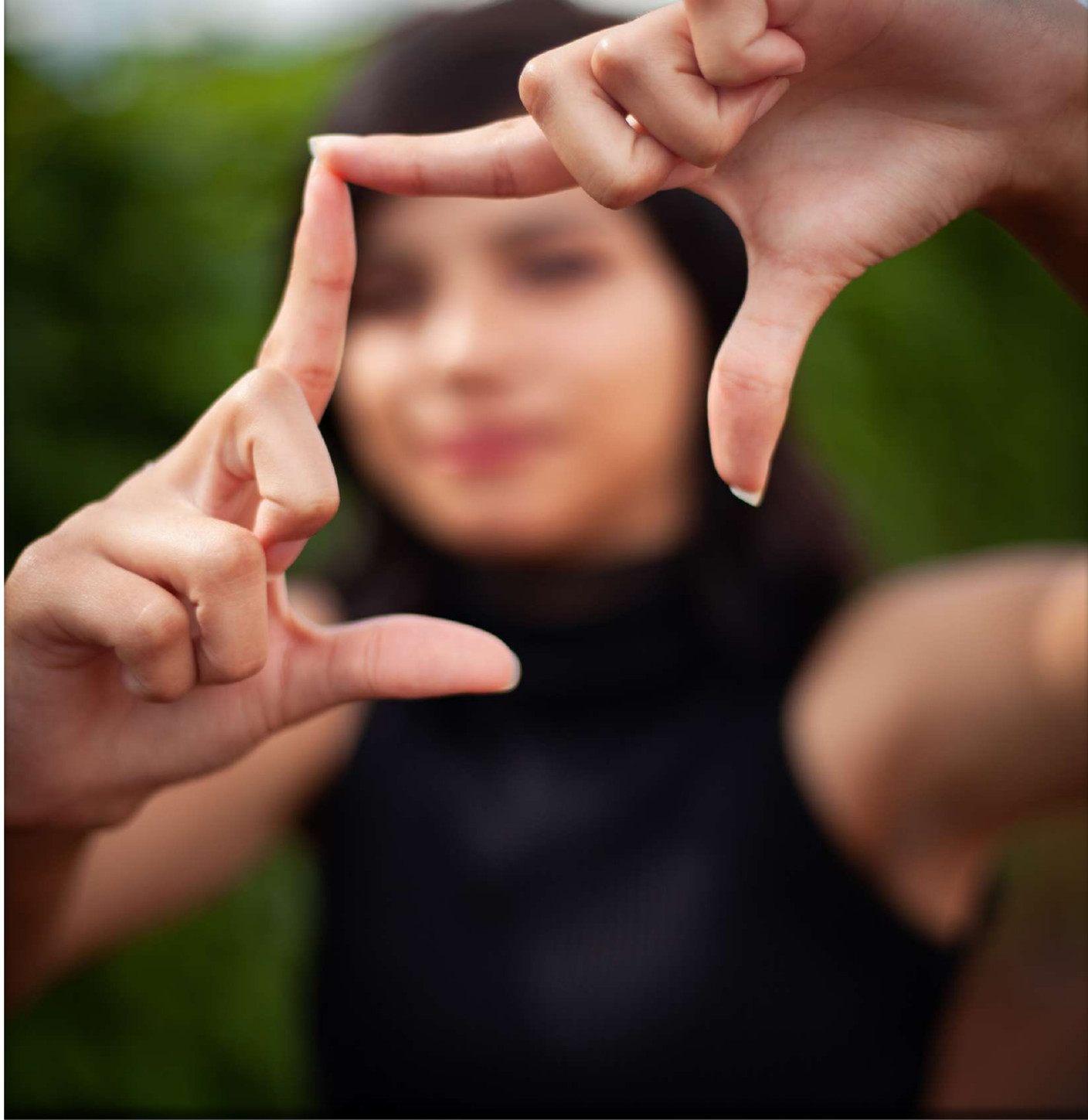
Facilitating

Facilitating is not just a skill, this becomes a **key piece** migrating to Quality Assistance model.

QA engineers must ensure that team have the right tools, right data, and environments they need to carry out their testing efficiently.

- QE will develop and maintain a test framework that everyone can use to write automated acceptance tests
- QE will take ownership of test environments and collaborate in IaC tools for those environment
- QE will collaborate in CI/CD tools to provide a platform to run unit tests
- QE will provide tools to review test results
- QE will offer choices to debug and reproduce test scenarios.

So in this model, what does a Quality Engineer actually do?



At a Story level

The QA engineers join the Kick off, so they can **help finding edge cases and risks.**

As outcome of this session, the team has been considered the main test scenarios, and they come up with the acceptance criteria for the story.

At a Feature level

For larger features (composed of many stories) that cut across multiple squads and components, a more holistic view of the solution is needed to test effectively.

The **developers still own testing of the individual stories**, but the **planning and coordinating of end to end testing** for such big sized features **is owned by the QA Engineers.**

At a Team level

With less time being spent on hands-on testing of stories, the QA Engineers now have time to spend on:

- working on complex scenarios
- thinking at solution layer, trying products integration
- creating automated test frameworks that other developers later add tests to
- improving the run time of automated tests (using parallelization or other techniques)
- looking into reducing flakiness of tests
- improving test data creation tools
- Improve test reports
- Identify weaker functional areas
- exploring ways of improving how we do things, etc

Let’s make change happen!

Óscar Barrios

obarrios@suse.com

Test Automation Specialist @SUSE Manager